BERT-Based Automatic Question Classification Using Deep Learning

M. Sai Sahasra
Roll No. 2420030706
Dept. of [Department]
[Institution Name]
[City], India
[email / ORCID]

Ch. Lucky Reddy
Roll No. 2420030739
Dept. of [Department]
[Institution Name]
[City], India
[email / ORCID]

K. Sai Krishna
Roll No. 2420030632
Dept. of [Department]
[Institution Name]
[City], India
[email / ORCID]

R. Vignesh Reddy
Roll No. 2420090006
Dept. of [Department]
[Institution Name]
[City], India
[email / ORCID]

***Abstract*—Automatic question classification, the task of assigning a question to a semantic category, is a key component of question answering systems, FAQ retrieval, online examination platforms and educational chatbots. Traditional approaches based on keywords, handcrafted lexical features and rule sets struggle with the many ways in which the same information need can be phrased. This paper presents a question classification system built on a pre-trained BERT model that is fine-tuned with a softmax classification layer to predict one of six question categories together with a confidence score. The pipeline covers data cleaning, WordPiece tokenization, transfer-learning-based fine-tuning, evaluation and deployment as a full-stack web application with a FastAPI back end, a React front end and a MongoDB history store. On a held-out test set of 2,196 questions the system reaches an accuracy of 0.94, a weighted precision and F1-score of 0.93, a weighted recall of 0.94 and an average inference time of 87.6 ms per question, which makes real-time use practical. Error analysis shows which categories remain most easily confused. The results indicate that contextual representations from BERT provide a reliable and deployable basis for automatic question classification.**

Index Terms*— *question classification, BERT, transformer, transfer learning, natural language processing, deep learning

I. Introduction

Online learning platforms, digital libraries, examination systems and chatbots receive thousands of questions every day. Sorting these questions into meaningful categories by hand is slow, inconsistent and demands considerable human effort. Question classification, which identifies the kind of information a question asks for, addresses this problem automatically. The category assigned to a question can be used to route it to the right expert or answer source, to narrow the search space of a question answering (QA) system, to organize an FAQ collection, and to improve search in educational websites [1, 2].

Early question classifiers relied on hand-built rules, bag-of-words features and support vector machines (SVMs) with lexical and syntactic features [1, 2]. Convolutional neural networks (CNNs) later removed part of the feature engineering effort and raised accuracy on the TREC benchmark [3, 4]. However, keyword-driven and feature-driven systems treat a question mostly as a collection of surface patterns. They have difficulty when the same intent is expressed with different words, or when a word changes its meaning with context, for example "*Which* country . . . " versus "*Which* year . . . ".

Bidirectional Encoder Representations from Transformers (BERT) [5] changed this situation. BERT is pre-trained on large unlabeled text corpora with a deep bidirectional Transformer encoder [6], so that every word is represented in the context of the whole sentence. A single additional output layer is enough to adapt the pre-trained model to a new classification task. Fine-tuning BERT therefore combines strong contextual understanding with a small amount of task-specific training [7].

This paper presents a complete question classification system based on BERT. The main contributions are as follows.

- A fine-tuned `bert-base-uncased` classifier that maps a natural-language question to one of six categories and returns a confidence score for the prediction.
- A reproducible pipeline covering dataset preparation, preprocessing, tokenization, fine-tuning, evaluation and error analysis.
- A full-stack deployment with a FastAPI inference service, a React dashboard and a MongoDB history store, which turns the trained model into a usable prototype for real-time prediction.
- An evaluation with accuracy, precision, recall, F1-score, a confusion matrix and inference-time measurement, together with an analysis of the remaining errors.

The rest of the paper is organized as follows. Section II reviews related work. Section III states the problem and the research gap. Section IV describes the methodology, Section V the system design and implementation, and Section VI the experimental results. Section VII discusses feasibility and limitations, and Section VIII concludes.

II. Related Work

A. Feature-Based and Rule-Based Classifiers

Li and Roth [1] introduced the widely used hierarchical question taxonomy, with six coarse and fifty fine classes, and the TREC question dataset that accompanies it. Their classifier used a hierarchical learning architecture over lexical, syntactic and semantic features. Zhang and Lee [2] compared several machine learning algorithms and found that an SVM using surface-text features and a tree kernel outperformed the other classical

methods. Nguyen and Le [8] improved SVM-based classification through feature extraction and selection, reporting 95.2% accuracy on coarse classes and 91.6% on fine classes. Madabushi and Lee [9] reached 97.2% on the fine-grained TREC set with hand-written rules built from question syntax and semantic concepts. These systems are accurate, but they depend on manually designed features or rules that are costly to maintain and may not transfer to unseen wording, domains or languages.

B. Neural Approaches

Kim [3] showed that a simple CNN over pre-trained word vectors achieves strong results on several sentence classification tasks, including 95.0% on TREC. Ma et al. [4] introduced dependency-based convolution and reported about 95.6% on TREC, at the price of requiring a dependency parser. CNNs capture local n-gram patterns well, but they do not model the full context of a word as directly as a Transformer encoder does.

C. Transformer-Based Approaches

Devlin et al. [5] introduced BERT and showed that fine-tuning a pre-trained deep bidirectional model improves a wide range of language understanding tasks. Sun et al. [7] studied how to fine-tune BERT for text classification and showed that the outcome depends on the fine-tuning strategy and on the quality and size of the training data. Sentence-BERT [10] produces semantically meaningful sentence embeddings, but it is designed mainly for similarity search rather than direct classification. For question classification specifically, Xu et al. [11] proposed BERT-QC and reported 96.2% on TREC with six classes and 92.0% with fifty classes, while also improving science-question classification and downstream QA. Mallikarjuna and Sivanesan [12] combined data augmentation with BERT-family models for limited labeled data and reported 98.11% on TREC. Aburass et al. [13] combined ELECTRA, GloVe and an LSTM in an ensemble and reported 80% accuracy on the TREC test set, showing that more complex ensembles do not automatically give better results. Table I summarizes these studies.

III. Problem Statement and Research Gap

Given a natural-language question q and a fixed set of categories $\mathscr{C} = \{c_1, \ldots, c_K\}$, the goal is to learn a function $f : q \mapsto (\hat{c}, s)$ that returns the most likely category $\hat{c} \in \mathscr{C}$ and a confidence score $s \in [0,1]$. In this work $K = 6$. The system should operate in real time and should be usable by non-expert users through a simple interface.

The literature review shows the following gaps.

- SVM, rule-based and TF-IDF systems depend on manually designed features and rules that capture complex language patterns poorly [1, 2, 9].
- CNN-based methods capture local patterns but model the contextual relationship between words only indirectly [3, 4].
- Several high-performing approaches need extra parsing, augmentation or complicated hybrid architectures, which raises implementation and computational cost [4, 12, 13].
- Many Transformer studies target specialized or domain-specific data, and few offer a simple end-to-end classifier together with evaluation, error analysis and a usable interface [11].
- Models are sensitive to variation in question wording, which calls for contextual rather than keyword-based understanding.

The proposed system addresses these gaps by fine-tuning a single pre-trained BERT model end to end, evaluating it with several metrics instead of accuracy alone, analyzing its errors, and exposing it through a real-time web application.

IV. Methodology

A. Overview

The method follows eight steps: data collection, data preprocessing, BERT tokenization, BERT representation, fine-tuning, prediction, confidence score calculation and result display. During training, labeled questions are tokenized and used to fine-tune the model, and the best checkpoint on the validation split is retained and evaluated on the held-out test set. During inference, a user question is passed through the same tokenizer and the trained model, and the predicted category and its confidence are returned to the front end.

B. Dataset and Preprocessing

The primary data source is the TREC question classification dataset [1], a public benchmark that is distributed with six coarse classes (abbreviation, entity, description, human, location and numeric) and in the commonly used version contains 5,500 training and 500 test questions. Labeled question collections available through the Hugging Face dataset hub were considered as complementary sources. The evaluation reported in Section VI uses six categories and a held-out test set of 2,196 questions.

Preprocessing consists of the following operations: (i) removing invalid and duplicate records and unnecessary characters, (ii) validating the text, (iii) mapping category names to integer labels, (iv) inspecting the class distribution for imbalance, and (v) splitting the data into training, validation and test sets. Because BERT was pre-trained on natural text, stop words are not removed and the words are not stemmed, so that the grammatical cues that signal the question type, such as interrogative words, are preserved.

C. Tokenization

Each question is converted to input for BERT with the WordPiece tokenizer of `bert-base-uncased`. The tokenizer lowercases the text, splits rare words into sub-word units, and adds the special tokens `[CLS]` at the beginning and `[SEP]` at the end. Sequences are padded or truncated to a fixed maximum length, and an attention mask marks which positions contain real tokens. The model therefore receives, for each question, a vector of token identifiers and a binary attention mask.

TABLE I
SUMMARY OF RELATED WORK ON QUESTION CLASSIFICATION

Study	Technique	Reported outcome	Limitation
Li and Roth [1]	Hierarchical feature-based classifier	Introduced the TREC taxonomy and benchmark	Relies on engineered lexical and semantic features
Zhang and Lee [2]	SVM with tree kernel	Best among the classical methods tested	Needs syntactic processing; no deep context
Kim [3]	CNN with word vectors	95.0% on TREC	Local patterns only; limited context modeling
Ma et al. [4]	Dependency-based CNN	About 95.6% on TREC	Requires dependency parsing
Nguyen and Le [8]	SVM with feature selection	95.2% coarse, 91.6% fine	Depends on manually selected features
Madabushi and Lee [9]	Syntax and semantic rules	97.2% on fine-grained TREC	Rules are hard to maintain and generalize
Xu et al. [11]	BERT-QC	96.2% (six classes), 92.0% (fifty classes)	Heavier than classical models; transfer across datasets is hard
Mallikarjuna and Sivanesan [12]	Augmentation with BERT-family models	98.11% on TREC with limited labels	Augmented text may add noise
Aburass et al. [13]	ELECTRA, GloVe and LSTM ensemble	80% on the TREC test set	High complexity and computational cost

Outcomes are as reported by the respective authors on their own experimental settings and are not directly comparable with one another.

D. BERT Encoder and Classification Layer

The pre-trained BERT-base encoder [5] consists of 12 Transformer layers with a hidden size of 768 and 12 self-attention heads. Every layer applies multi-head self-attention, in which the representation of each token is a weighted combination of all tokens in the question:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V, \tag{1}$$

where Q, K and V are the query, key and value matrices and d_k is the key dimension [6]. Because attention runs in both directions, each word is interpreted using the words on its left and its right.

The final hidden state of the `[CLS]` token, $\mathbf{h}_{\text{cls}} \in \mathbb{R}^{768}$, serves as the representation of the whole question. A linear layer followed by a softmax produces the class probabilities:

$$\mathbf{z} = \mathbf{W}\mathbf{h}_{\text{cls}} + \mathbf{b}, \qquad p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \tag{2}$$

with $\mathbf{W} \in \mathbb{R}^{K\times 768}$ and $K = 6$. The predicted category is $\hat{c} = \arg\max_k p_k$.

E. Fine-Tuning

Transfer learning is used instead of training from scratch. All parameters of the pre-trained encoder and the new classification layer are updated jointly on the labeled questions by minimizing the cross-entropy loss:

$$\mathscr{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log p_{ik}, \tag{3}$$

where N is the number of training questions and y_{ik} is the one-hot label. The model is implemented in PyTorch [14] with the Hugging Face Transformers library [15]. The learning rate, batch size, number of epochs and maximum sequence length follow the standard BERT fine-tuning recipe [5, 7]. The validation split is used to monitor training and to keep the best checkpoint.

F. Confidence Score

The confidence score reported for a prediction is the highest softmax probability, $s = \max_k p_k$. The full probability distribution is also returned, so that the user can see how strongly the other categories compete. The score is a model probability and not a guarantee of correctness; for ambiguous questions it can be moderate even when the top category is right.

V. System Design and Implementation

A. Architecture

The trained model is deployed as a layered web application, shown in Fig. 1. The *user interface* is a React single-page application. The *API server* receives the question over HTTP as JSON, applies authentication and routing, and forwards the text to the *BERT model layer*, which tokenizes it and performs a forward pass through the fine-tuned model. The prediction and its confidence are returned to the interface and, together with a timestamp and the user identifier, stored in the *database layer* so that a user can review the classification history.

B. Modules

The implementation is organized into six modules.

1. **Data preparation:** collects the labeled questions, cleans invalid and duplicate records, prepares the labels and creates the training and test splits.

2. **Preprocessing:** converts questions to token identifiers and attention masks with padding and truncation.

3. **Model training:** loads `bert-base-uncased`, adds the classification layer, trains with cross-entropy loss in PyTorch and saves the model and tokenizer.

4. **Inference:** loads the saved model, tokenizes a new question, runs the forward pass, computes class probabilities and selects the category.

5. **API and integration:** a FastAPI service exposes `/api/classify`, `/api/history` and `/api/auth`, pro-

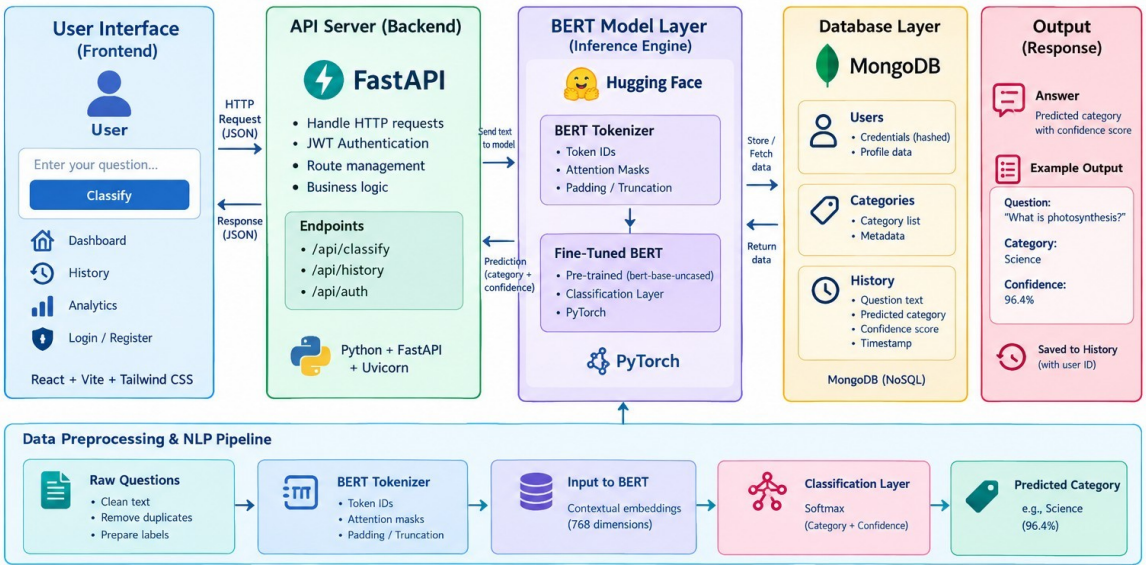


Fig. 1. System design of the proposed BERT-based question classification application, from the user interface through the FastAPI server, the BERT inference engine and the MongoDB store, with the data preprocessing pipeline underneath.

TABLE II
TECHNOLOGY STACK

Component	Technology
Frontend	React 19, Vite, React Router
Styling and UI	Tailwind CSS, Lucide React
Backend	Python 3.11+, FastAPI
NLP model	BERT (`bert-base-uncased`), Hugging Face Transformers
Deep learning	PyTorch
Database	MongoDB
Security and visualization	JWT, bcrypt, Recharts
Dataset	TREC question classification dataset

tected with JSON Web Token (JWT) authentication; passwords are stored hashed with bcrypt.

6. **Response and storage:** the React interface shows the category and confidence, while MongoDB stores users, categories and classification history for analytics.

C. Technology Stack

Table II lists the tools used. All of them are open source, and the dataset is freely available, so the system can be developed and run without specialized hardware or licensing cost.

D. User Interface

The dashboard (Fig. 2) lets a user type or choose a sample question and press a single button to classify it. The result panel shows the predicted category, the confidence score and a bar chart of the probability distribution, and a recent-predictions list records previous queries. Further pages present model evaluation, model comparison, error analysis, dataset information and a project description.

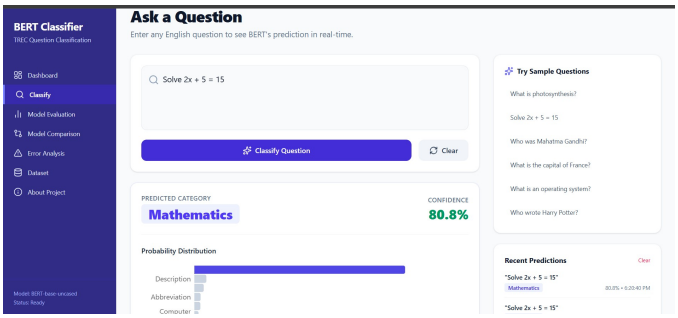


Fig. 2. Classification page of the web dashboard: question input, sample questions, predicted category with confidence, probability distribution and recent predictions.

VI. EXPERIMENTAL RESULTS

A. Evaluation Metrics

The model is evaluated on the held-out test set with accuracy, precision, recall and F1-score. For class k, with true positives TP_k, false positives FP_k and false negatives FN_k,

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = \frac{2P_kR_k}{P_k + R_k}. \quad (4)$$

Accuracy is the fraction of correctly classified questions. Weighted averages weight each class by its number of test questions (support), and macro averages weight all classes equally. A confusion matrix is used to see which categories are mistaken for each other, and the average time needed to classify one question measures the suitability for real-time use.

B. Overall Performance

Table III gives the overall results on the 2,196 test questions. The model reaches an accuracy of 0.94, and the weighted precision, recall and F1-score are all at least 0.93, which indicates that the model is not favoring one metric at the expense of an-

TABLE III
OVERALL TEST PERFORMANCE

Metric	Value
Test accuracy	0.94
Precision (weighted)	0.93
Recall (weighted)	0.94
F1-score (weighted)	0.93
Average inference time	87.6 ms
Test questions	2,196

TABLE IV
PER-CLASS CLASSIFICATION REPORT

Category	Precision	Recall	F1-score	Support
Science	0.96	0.95	0.95	412
Technology	0.92	0.91	0.91	387
Geography	0.94	0.93	0.93	368
Sports	0.90	0.89	0.89	342
Business	0.93	0.94	0.93	351
Entertainment	0.91	0.92	0.91	336
Macro average	0.93	0.92	0.92	2196
Weighted average	0.94	0.94	0.93	2196

other. The average inference time of 87.6 ms per question is well below the delay that users perceive as interactive, so the model can serve online requests without batching or a dedicated accelerator.

C. Per-Class Performance

Table IV and Fig. 3 give the per-class results. All six categories obtain an F1-score of at least 0.89. Science is classified best, with precision 0.96, recall 0.95 and F1-score 0.95. Sports has the lowest scores (precision 0.90, recall 0.89, F1-score 0.89), followed by Technology and Entertainment with F1-scores of 0.91. The macro averages of 0.93 precision, 0.92 recall and 0.92 F1-score are close to the weighted averages, so the performance is not driven by the larger classes alone.

D. Confusion Matrix and Error Analysis

The confusion matrix in Fig. 4 shows a strong diagonal and only a small number of errors in each row. The largest single confusion is that 16 Technology questions are predicted as Science, followed by 12 Science questions predicted as Technology and 12 Sports questions predicted as Business. These pairs share vocabulary and subject matter: scientific and technological questions often mention the same concepts, and sports questions about clubs, sponsorship or prize money have a business flavor. Sports questions are also spread over several other classes, such as Science (8) and Geography (7), which agrees with their lower recall. Errors of this kind are not purely model failures. In many cases the question belongs legitimately to more than one category, and the label reflects a single annotator decision. For such questions the probability distribution returned by the application shows that the competing category has non-negligible probability, which a user or a downstream system can use to decide whether to accept the prediction.

E. Comparison with Reported Results

Published accuracies on TREC range from 95.0% for a CNN [3] through 96.2% for BERT-QC [11] to 98.11% with data augmentation [12] (Table I). These values, and the 0.94 accuracy

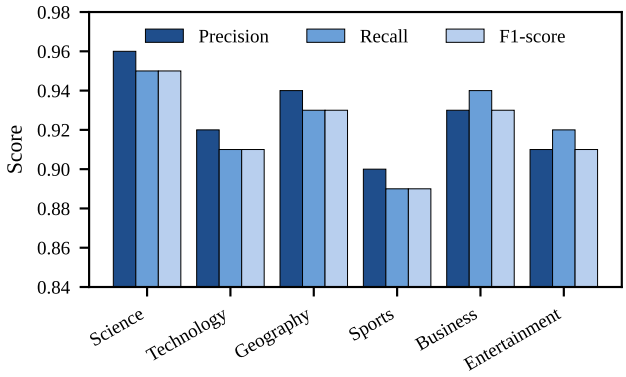


Fig. 3. Precision, recall and F1-score for each category on the test set.

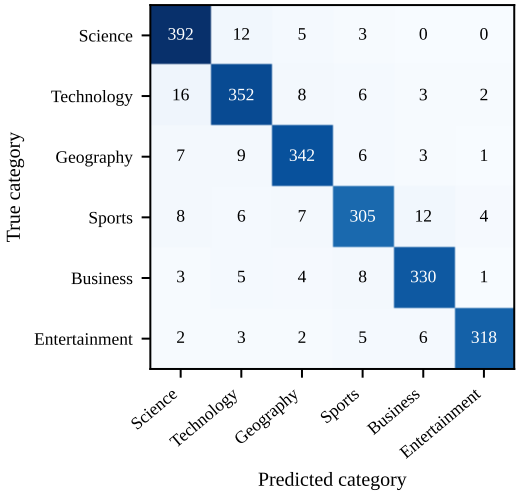


Fig. 4. Confusion matrix of the fine-tuned BERT classifier on the 2,196 test questions.

reported here, were obtained with different category sets, data sizes and splits, and they should not be ranked against each other directly. The relevant conclusion is that a single fine-tuned BERT-base model, without parsers, hand-written rules, augmentation or ensembles, reaches competitive accuracy, and that it does so while remaining fast enough for real-time use. A controlled comparison with a TF-IDF baseline and with DistilBERT, RoBERTa and DeBERTa on the same split is left as future work.

VII. DISCUSSION

A. Feasibility

Technically, the system uses open-source tools (Python, PyTorch, Hugging Face Transformers, FastAPI and React) and a public dataset, and it can be developed and tested on a standard laptop. *Economically*, no commercial software or special hardware is required, so implementation and maintenance costs are low. *Operationally*, the web interface is simple to use, and a prediction is returned within a fraction of a second, which suits educational platforms, FAQ systems, search engines and chat-

bots.

B. Innovation and Novelty

BERT itself is an established model, so the contribution lies in the integration. The work combines contextual BERT representations, transfer learning, confidence-aware prediction, error analysis and a deployed real-time application into one reproducible, student-scale pipeline that addresses the limitations identified in Section III. Showing the probability distribution and the confusion matrix makes the behavior of the model inspectable, which helps users to judge each prediction.

C. Limitations

The current system has the following limitations. It handles English questions only and classifies into a fixed set of predefined categories, so questions from other domains or languages may be misclassified. BERT needs more memory and computation than classical models, although the measured latency is acceptable. Confidence scores are not calibrated probabilities. Finally, the evaluation uses a single data split, and a comparison with other baselines and multiple runs would give stronger statistical evidence.

VIII. Conclusion and Future Work

This paper presented an automatic question classification system based on a fine-tuned BERT model. The model uses the contextual representation of the `[CLS]` token and a softmax layer to predict a question category and a confidence score, and it is deployed through a FastAPI, React and MongoDB application. On a test set of 2,196 questions it achieved an accuracy of 0.94, weighted precision and F1-score of 0.93, weighted recall of 0.94 and an average inference time of 87.6 ms. The confusion analysis shows that the remaining errors occur mainly between categories with overlapping vocabulary, such as Technology and Science.

Future work includes (i) controlled comparison with a TF-IDF baseline and with DistilBERT, RoBERTa and DeBERTa to study the trade-off between accuracy and efficiency, (ii) fine-grained classification with the fifty TREC subclasses, (iii) data augmentation for limited-data settings, (iv) multilingual support, (v) calibration of the confidence scores, (vi) attention-based explanations of predictions, and (vii) integration with a complete question answering system.

Acknowledgment

The authors thank their course guide, Ms. Swanthana, and the Applied Machine Learning for Text Analysis course (24ALT3101) for guidance and support during this project.

References

[1] X. Li and D. Roth, "Learning question classifiers," in *Proc. 19th Int. Conf. Computational Linguistics (COLING)*, 2002, pp. 556–562.

[2] D. Zhang and W. S. Lee, "Question classification using support vector machines," in *Proc. 26th Annu. Int. ACM SIGIR Conf.*, 2003, pp. 26–32.

[3] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.

[4] M. Ma, L. Huang, B. Zhou, and B. Xiang, "Dependency-based convolutional neural networks for sentence embedding," in *Proc. 53rd Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2015, pp. 174–179.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[6] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[7] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune BERT for text classification?" in *Chinese Computational Linguistics (CCL)*, 2019, pp. 194–206.

[8] V.-T. Nguyen and A.-C. Le, "Improving question classification by feature extraction and selection," *Indian J. Sci. Technol.*, 2016.

[9] H. T. Madabushi and M. Lee, "High accuracy rule-based question classification using question syntax and semantics," in *Proc. 26th Int. Conf. Computational Linguistics (COLING)*, 2016, pp. 1220–1230.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[11] D. Xu, V. Jansen, J. Martin, Z. Xie, P. Jansen, and P. Clark, "Multi-class hierarchical question classification for multiple choice science exams," in *Proc. 12th Language Resources and Evaluation Conf. (LREC)*, 2020.

[12] M. Mallikarjuna and A. Sivanesan, "Question classification using limited labelled data," *Inf. Process. Manage.*, 2022.

[13] S. Aburass, O. Dorgham, and M. Abu Rumman, "An ensemble approach to question classification: Integrating Electra transformer, GloVe, and LSTM," arXiv:2308.06828, 2023.

[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[15] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.